

1. Introduction

This assignment uses linear and logistic regression to not only predict survival but also to investigate factors influencing passenger fare, age, passenger class, and family size. This dataset contains 12 attributes including passengerID, #survived- which shows whether they survived or not (0 for No and 1 for yes), #Pclass which shows the ticket classes that range from 1st class to 3rd class, Name of the passenger, Gender, Age, Number of Siblings or spouses aboard, number of parents or children aboard, ticket number, passenger fare, cabin number and port of embarkation

2. Analysis with Linear and Logistic Regression Models, Visualizations

Data overview

Key numerical statistics from df.describe():

Average age: 29.7 years (SD=14.5)

38.4% overall survival rate

Average fare: £32.20 (wide variance from £0 to £512)

2.1 Logistic Regression

Logistic regression is a technique used to predict categorical variables. Based on this dataset I predicted the variables survival, the class to which the passengers belonged, embarkation port, and family status (Whether they were alone or with family on board) using several features from the data set

2.1.1 Predicting survival based on the passenger class, age, sex, and fare

```
#Predicting survival based on pclass, age, sex, fare using logistic regression
survival_features = ['Pclass', 'Sex', 'Age', 'Fare']
X = df[survival_features]
y = df['Survived']

model = LogisticRegression(max_iter=1000)
model.fit(X, y)

coeff_df = pd.DataFrame({
    'Feature': survival_features,
    'Coefficient': model.coef_[0],
    'Odds_Ratio': np.exp(model.coef_[0])
})
```

```
y_predict_survival=model.predict(X)
```

+ Code

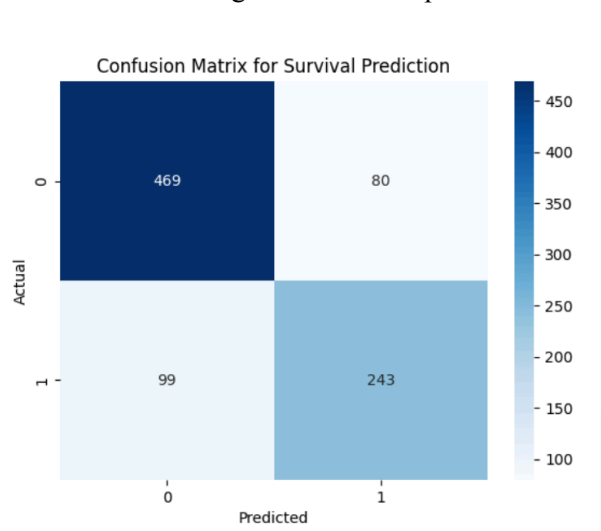
+ Markdown

```
from sklearn.metrics import accuracy_score, confusion_matrix

accuracy_survival=accuracy_score(y,y_predict_survival)
print (accuracy_survival)
```

0.7991021324354658

The accuracy of this model shows that the model correctly classifies about 80% of the instances in the dataset and can be used to predict whether a passenger survived or not. Through this model I input random features to predict the survival of the passenger. **(Appendix)**. The confusion matrix for this model compares the actual outcomes with the predicted outcomes. The model has 469 true negatives meaning it correctly predicted 469 individuals as not surviving. And it predicted 243 individuals as surviving as can be seen in the bottom right corner. However the model incorrectly predicted 80 individuals as surviving when they did not and 99 individuals as surviving when they actually survived. Based on the total records of 891 passengers, this model is of high accuracy despite the false positives and negatives which are way less than the true negatives and true positives.



confusion matrix survival

2.1.2 Predicting passenger class using fare, family size, age

```

]: #Predicting passenger class using fare, family size, age, logistic regression
X_pclass = df[["Fare", "FamilySize", "Age", "Embarked"]]
y_pclass = df["Pclass"]
log_reg_pclass = LogisticRegression(multi_class='ovr', max_iter=200)
log_reg_pclass.fit(X_pclass, y_pclass)
y_pred_pclass = log_reg_pclass.predict(X_pclass)
accuracy_pclass = accuracy_score(y_pclass, y_pred_pclass)
print(accuracy_pclass)

```

0.792368125701459

The accuracy of this model shows that the model correctly classifies about 79% of the instances in the dataset and can be used to predict what a passenger's class was. Through this model I input random features to predict the passenger class. **(Appendix)**. The confusion matrix **(Appendix)** compares the actual passenger classes (rows) with the predicted classes (columns). 207 passengers in 1st class were correctly predicted as such, 17 passengers in 2nd class were correctly predicted as such, and 482 passengers in 3rd class were correctly predicted as such. Off diagonal cells represent misclassifications, meaning passengers were wrongly classified. 8 (row 0, column 2): Eight passengers actually in 1st class were misclassified as being in 3rd class. 15 (row 1, column 0): Fifteen passengers actually in 2nd class were misclassified as being in 1st class.

Structure of the Confusion Matrix

Rows: Represent the actual classes (ground truth).

Row labeled "0": Passengers actually in 1st class.

Row labeled "1": Passengers actually in 2nd class.

Row labeled "2": Passengers actually in 3rd class.

Columns: Represent the predicted classes.

Column labeled "0": Predicted as 1st class.

Column labeled "1": Predicted as 2nd class.

Column labeled "2": Predicted as 3rd class.

confusion matrix fare

2.1.3 Predicting Embarkation Port using class, fare, age and family size

For this model, I had to create a new column named family size using the number of siblings, spouse, parents, and or children aboard.

The accuracy of this model shows that the model correctly classifies about 73% of the instances in the dataset and can be used to predict what a passenger's embarkation port was. Through this model I input random features to predict the embarkation port. (Appendix 1.2). Additionally, the embarkation ports were converted to numerical- 0 for S, 1 for C, and 2 for Q where C = Cherbourg, Q = Queenstown, and S = Southampton. The below ROC curve evaluates the performance of the prediction model. Each curve corresponds to one of the three classes (ports), and the Area Under the Curve (AUC) values indicate how well the model distinguishes between the classes.

- **Class 0 (Blue Curve):** The AUC for this class is 0.62, which suggests that the model has limited ability to distinguish passengers who embarked at this port from those who embarked at other ports. This performance is only slightly better than random guessing (AUC = 0.5).
- **Class 1 (Red Curve):** The AUC for this class is 0.67, showing moderate discriminatory power for identifying passengers from this port. While better than Class 0, it still indicates room for improvement.
- **Class 2 (Green Curve):** The AUC for this class is 0.75, which demonstrates relatively good performance in distinguishing passengers who embarked at this port compared to the others. This is the strongest-performing class in the model.

The ROC curves and AUC values suggest that the model performs best for Class 2 but struggles with Classes 0 and 1, particularly Class 0, where its performance is close to random guessing.

3. Linear Regression

Linear regression is a technique used to predict continuous variables. Based on this dataset, I used linear regression to predict fare and age.

3.1 Predicting Fare

The relatively high MSE and RMSE suggest that there are significant prediction errors for fare. The R² value of 0.379 (**Appendix**) shows that the model explains only a moderate portion of the variance in fare. This suggests:

- The predictors used in the model have some relevance but are not sufficient to capture all factors influencing fare.

3.2 Predicting Age

The high MSE and RMSE suggest that there are significant errors in predicting age, with an average deviation of nearly 12 years. **(Appendix)** The low r^2 value indicates that the model does not explain much of the variability in age. This suggests that:

- The relationship between predictors and age is weak.
- Important features influencing age are missing from the model, age was one of the attributes with missing values

4. Summary

Logistic Regression Models

Survival Prediction

The logistic regression model predicting survival based on passenger class, age, sex, and fare achieved an accuracy of approximately 80%. This indicates that the model is fairly reliable in predicting passenger survival. Key observations include:

The model correctly predicted 469 non-survivors and 243 survivors.

There were 80 false positives and 99 false negatives.

The high accuracy suggests that these features (class, age, sex, and fare) are strong predictors of survival.

Passenger Class Prediction

A logistic regression model was used to predict passenger class based on fare, family size, age, and embarkation port. This model achieved an accuracy of about 79%. Notable findings include:

The model was particularly accurate in predicting 1st and 3rd class passengers.

There was some confusion between 2nd and 3rd class predictions.

The high accuracy indicates that these features, especially fare, are good indicators of passenger class.

Embarkation Port Prediction

The model predicting embarkation port using class, fare, age, and family size had an accuracy of approximately 73%. The ROC curve analysis revealed:

Best performance for predicting one port (likely Southampton) with an AUC of 0.75.

Moderate performance for another port (AUC of 0.67).

Poor performance for the third port (AUC of 0.62), barely better than random guessing.

Linear Regression Models

Fare Prediction

The linear regression model for predicting fare showed moderate performance:

The R^2 value of 0.379 suggests that the model explains only about 38% of the variance in fare.

High MSE and RMSE values indicate significant prediction errors.

This implies that while the chosen features have some predictive power, other important factors influencing fare are likely missing from the model.

Age Prediction

The linear regression model for predicting age performed poorly:

High MSE and RMSE values suggest significant errors in age prediction, with an average deviation of nearly 12 years.

A low R^2 value indicates that the model explains very little of the variability in age.

This suggests that either the relationship between the predictors and age is weak, or crucial features influencing age are missing from the model.

Conclusion

- Survival on the Titanic was strongly influenced by passenger class, age, sex, and fare, with the model achieving high predictive accuracy.
- Passenger class can be reliably predicted from fare, family size, age, and embarkation port, indicating a strong relationship between these factors and socioeconomic status.
- Embarkation port prediction is moderately successful, with better performance for one port (likely the most common one) than the others.
- Fare prediction using linear regression shows moderate success, while some factors like class and family size influence fare, other important variables may be missing from the model.
- Age prediction using the available features is unreliable, age may be largely independent of the other variables in the dataset or that non-linear relationships exist.

APPENDIX

LOGISTIC REGRESSIONS

Predicting survival

```
# Predict Survival if Pclass is 1,sex is female, age is 20 and fare is 13
new_passenger = np.array([[1, 1, 20, 13]]) # [Pclass, Sex, Age, Fare]

# Predict survival
predicted_survival = model.predict(new_passenger)

# Output prediction result
print("Predicted Survival:", "Survived" if predicted_survival[0] == 1 else "Did Not Survive")
```

1. Predicted Survival: Survived

Predicting passenger class

```
]: #Predicting passenger class using fare,family size,age, logistic regression
X_pclass = df[["Fare", "FamilySize","Age","Embarked"]]
y_pclass = df["Pclass"]
log_reg_pclass = LogisticRegression(multi_class='ovr', max_iter=200)
log_reg_pclass.fit(X_pclass, y_pclass)
y_pred_pclass = log_reg_pclass.predict(X_pclass)
accuracy_pclass = accuracy_score(y_pclass, y_pred_pclass)
print(accuracy_pclass)
```

0.792368125701459

```
#Predicting passenger class if fare is 20,Family size is 3, Age is 15 and Embarked from C
new_passenger_1 = np.array([[20, 3, 15, 1]])

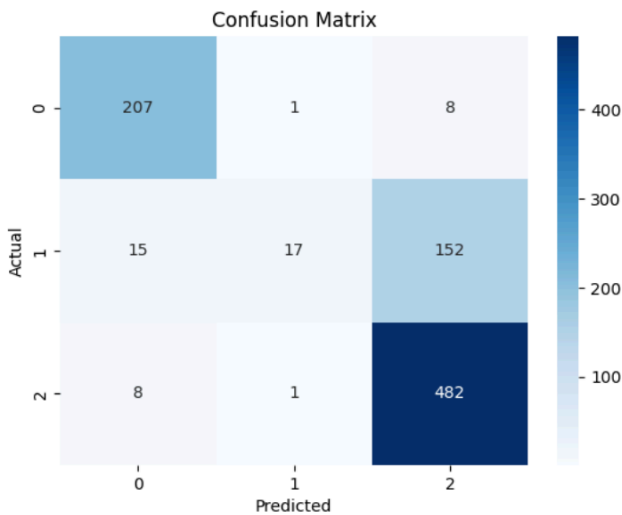
# Predict Class
predicted_class= log_reg_pclass.predict(new_passenger_1)

# Output prediction result
print("Predicted class is:",predicted_class)
```

Predicted class is: [3]

Confusion Matrix:

```
[[207  1  8]
 [ 15 17 152]
 [  8  1 482]]
```



Predicting Family Status

```
#Predict is alone or with family if Pclass is 2, Fare is 25, Age is 30, Embarked 1= queenstown
new_passenger_family = np.array([[2, 25, 30, 1]]) # [Pclass, Fare, Age, Embarked]

# Predict family status (IsAlone: 1 = Alone, 0 = With Family)
predicted_family_status = log_reg_family.predict(new_passenger_family)

# Output prediction result
print("Predicted Family Status:", "Alone" if predicted_family_status[0] == 1 else "With Family")
```

Predicting embarkation port

```
#Predicting Embarkation Port- Logistic Regression
X_embark = df[["Pclass", "Fare", "Age", "FamilySize"]]
y_embark = df["Embarked"]
log_reg_embark = LogisticRegression(multi_class='ovr', max_iter=200)
log_reg_embark.fit(X_embark, y_embark)
y_pred_embark = log_reg_embark.predict(X_embark)
accuracy_embark = accuracy_score(y_embark, y_pred_embark)
print(accuracy_embark)
```

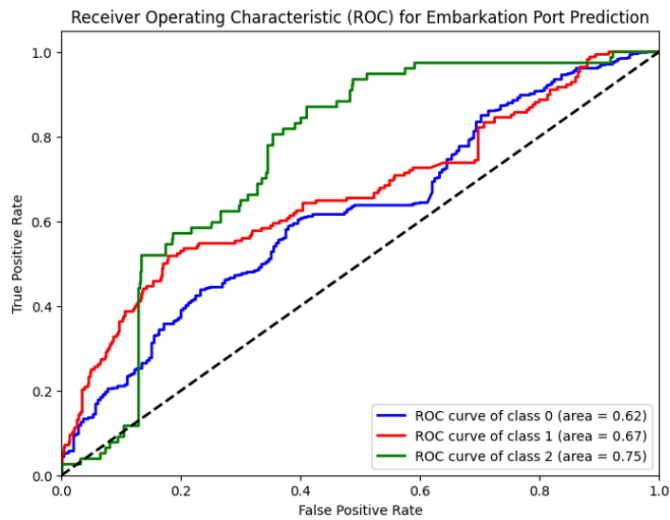
0.7306397306397306

```
[34]: # Predict Embarkation Port if Pclass is 3, age is 20, fare is 25, FamilySize is 2
new_passenger_2 = np.array([[3, 25, 20, 2]])

# Predict survival
predicted_embarkationport = log_reg_embark.predict(new_passenger_2)

# Output prediction result
print("Embarkation Port", predicted_embarkationport)
```

Embarkation Port [0]



LINEAR REGRESSIONS

Predicting fare

Model

```
### Linear Regression (Predicting Continuous Variables)

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Predicting Fare
X_fare = df[["Pclass", "Age", "FamilySize", "Embarked"]]
y_fare = df["Fare"]
lin_reg_fare = LinearRegression()
lin_reg_fare.fit(X_fare, y_fare)
y_pred_fare = lin_reg_fare.predict(X_fare)
mse_fare = mean_squared_error(y_fare, y_pred_fare)
print(f"MSE for Fare Prediction: {mse_fare:.2f}")
```

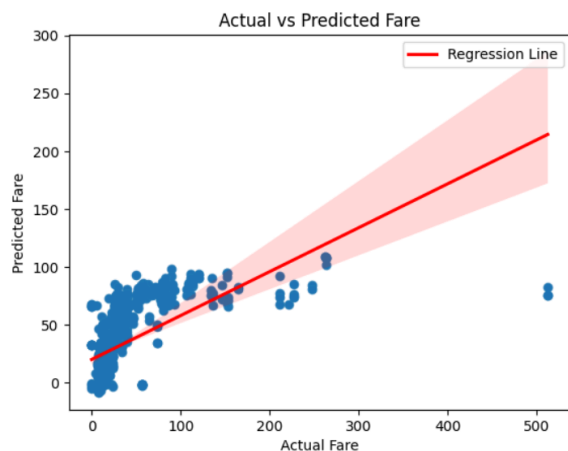
MSE for Fare Prediction: 1530.48

```
#Evaluating model
from sklearn.metrics import mean_squared_error, r2_score
mse_fare = mean_squared_error(y_fare, y_pred_fare)
rmse_fare = np.sqrt(mse_fare)
r2_fare = r2_score(y_fare, y_pred_fare)

print(f"Mean Squared Error (Fare): {mse_fare}")
print(f"Root Mean Squared Error (Fare): {rmse_fare}")
print(f"R-squared (Fare): {r2_fare}")
```

Mean Squared Error (Fare): 1530.4755890407496
Root Mean Squared Error (Fare): 39.12129329458255
R-squared (Fare): 0.3795365825308331

Visualization



Predicting Age

Model

```
# Predicting Age
X_age = df[["Pclass", "Fare", "FamilySize", "Embarked"]]
y_age = df["Age"]
lin_reg_age = LinearRegression()
lin_reg_age.fit(X_age, y_age)
y_pred_age = lin_reg_age.predict(X_age)
mse_age = mean_squared_error(y_age, y_pred_age)
print(f"MSE for Age Prediction: {mse_age:.2f}")
```

MSE for Age Prediction: 140.99

+ Code

+ Markdown

```
#Evaluating model
```

```
mse_age = mean_squared_error(y_age, y_pred_age)
rmse_age = np.sqrt(mse_age)
r2_age = r2_score(y_age, y_pred_age)

print(f"Mean Squared Error (Fare): {mse_age}")
print(f"Root Mean Squared Error (Fare): {rmse_age}")
print(f"R-squared (Fare): {r2_age}")
```

Mean Squared Error (Fare): 140.98542298432199
Root Mean Squared Error (Fare): 11.873728268085049
R-squared (Fare): 0.16735441336435197

Visualization

